

Full Stack Development with MERN

Project Documentation

Nutrition Assistant

A Personalized Nutrition Tracking & Recommendation Web App

1. Introduction

Project Title

NutriAssist — Personalized Nutrition Assistant

Team Members

The table below lists project contributors and their roles.

Name	Role	Contribution Area
V.Yoga Priya P.Chethana Reddy	Team Lead Full Stack Developer	Frontend, Backend Backend, Database

2. Project Overview

Purpose

NutriAssist helps users make healthier, more informed food choices by combining meal logging, nutrient tracking, and personalized recommendations in a single web application. Users log what they eat, see how it compares to calorie and macro targets calculated from their own body metrics and goals, and get recipe suggestions and grocery lists to help them act on those targets.

Key Features

- User authentication (register/login) with a health profile: age, gender, height, weight, activity level, goal, and dietary restrictions.
- Meal logging against a searchable food database, with quantities scaled per gram.
- A tracking dashboard showing today's calories/macros vs. targets, plus a 7-day calorie chart.
- Personalized recommendations: BMR (Mifflin-St Jeor) and TDEE calculation, with goal-adjusted daily calorie and macro (protein/carbs/fat) targets.
- Recipe suggestions filtered by meal type and goal, with a grocery list auto-generated from selected recipes.
- Educational resources: articles on nutrition fundamentals and a directory of nutrition professionals.

3. Architecture

Frontend (React)

Built with React (via Vite) and styled with Tailwind CSS. Client-side routing is handled by React Router, HTTP calls by Axios, and charts by Recharts. Icons come from lucide-react.

- pages/ — one component per route: Landing, Login, Register, Dashboard, MealLogPage, ProfileSettings, Recommendations, Recipes, Resources.

- components/ — shared UI: Navbar (with responsive mobile menu) and ProtectedRoute (route guard for authenticated pages).
- context/AuthContext.jsx — global auth state (current user, login/logout), backed by a JWT stored in localStorage.
- utils/api.js — a preconfigured Axios instance that attaches the JWT to every request.

Backend (Node.js + Express)

The backend follows a layered structure: routes define endpoints and attach middleware, controllers hold the business logic, and models define the MongoDB schemas via Mongoose.

- routes/ — authRoutes, foodRoutes, mealLogRoutes, recipeRoutes.
- controllers/ — authController, foodController, mealLogController, recipeController.
- middleware/authMiddleware.js — verifies the JWT on protected routes and attaches req.user.
- utils/nutritionCalc.js — shared BMR/TDEE/macro-target formulas used by the recommendations feature.
- seed/ — seedFoods.js and seedRecipes.js populate starter data for local development.

Database (MongoDB)

Four collections back the application:

Collection	Purpose & Key Fields
User	Login credentials (hashed password) plus health profile (age, gender, heightCm, weightKg, activityLevel, goal, dietaryRestrictions) and computed targets (bmr, tdee, dailyCalorieGoal, proteinTargetG, carbsTargetG, fatTargetG).
Food	A reference food database: name, category, nutrition values per 100g (calories/protein/carbs/fat/fiber), and dietary tags. Indexed for text search.
MealLog	One entry per food logged by a user: user & food references, a date-snapshotted foodName, quantityGrams, mealType, date ("YYYY-MM-DD"), and computed calories/protein/carbs/fat for that quantity.
Recipe	Suggested recipes: name, description, mealType, tags (diet), goalTags, prepTimeMinutes, servings, per-serving nutrition, ingredients array, and instructions.

4. Setup Instructions

Prerequisites


```

|   ├── utils/          api.js (Axios instance)
|   ├── App.jsx         Route definitions
|   └── main.jsx        App entry point
└── tailwind.config.js
└── package.json

```

Server (Node.js backend)

```

server/
├── config/            db.js – MongoDB connection
├── models/           User.js, Food.js, MealLog.js, Recipe.js
├── controllers/      authController, foodController,
│                     mealLogController, recipeController
├── routes/           authRoutes, foodRoutes,
│                     mealLogRoutes, recipeRoutes
├── middleware/       authMiddleware.js – JWT verification
├── utils/            nutritionCalc.js – BMR/TDEE/macro formulas
├── seed/             seedFoods.js, seedRecipes.js
└── server.js         App entry point

```

6. Running the Application

Run the backend and frontend in two separate terminals.

Component	Command (from its folder)	Default URL
Backend (server/)	npm run dev	http://localhost:5000
Frontend (client/)	npm run dev	http://localhost:5173

Once both are running, open the frontend URL in a browser, register an account, and use the app end to end.

7. API Documentation

All endpoints are prefixed with /api. Endpoints marked "Yes" under Auth require a valid JWT in the Authorization: Bearer <token> header.

Auth routes — /api/auth

Method	Endpoint	Auth	Description
POST	/register	No	Create a new user account. Body: name, email, password.
POST	/login	No	Authenticate and receive a JWT. Body:

			email, password.
GET	/profile	Yes	Get the logged-in user's full profile.
PUT	/profile	Yes	Update profile fields; automatically recalculates BMR/TDEE/macro targets.
GET	/targets	Yes	Preview calculated targets from the saved profile, without persisting.
POST	/targets	Yes	Calculate and save BMR/TDEE/calorie/macro targets to the profile.

Food routes — /api/foods

Method	Endpoint	Auth	Description
GET	/	Yes	Search/list foods from the reference database (used when logging a meal).

Meal log routes — /api/meallogs

Method	Endpoint	Auth	Description
POST	/	Yes	Log a food entry. Body: foodId, quantityGrams, mealType, date.
GET	/?date=YYYY-MM-DD	Yes	Get all entries and totals for a given date (defaults to today).
GET	/weekly	Yes	Get zero-filled daily totals for the 7 days ending on an optional endDate.
DELETE	/:id	Yes	Delete a logged entry.

Recipe routes — /api/recipes

Method	Endpoint	Auth	Description
GET	/? mealType=&goal=&diet=&search= =	Yes	List recipes, optionally filtered; defaults goal to the user's own profile goal.
POST	/grocery-list	Yes	Body: items: [{ recipeId, servings }]. Aggregates ingredients across selected recipes into one grocery list.

Example: Register

POST /api/auth/register

```
{ "name": "Jane Doe", "email": "jane@example.com", "password": "secret123" }
```

Example response:

```
{ "_id": "...", "name": "Jane Doe", "email": "jane@example.com", "token": "<JWT>" }
```

Example: Log a Meal

POST /api/meallogs

```
{ "foodId": "665...", "quantityGrams": 150, "mealType": "lunch", "date": "2026-07-14" }
```

Example response:

```
{ "log": { "foodName": "Chicken breast", "calories": 248, "protein": 46 }, "totals": { "calories": 248, "protein": 46 } }
```

8. Authentication

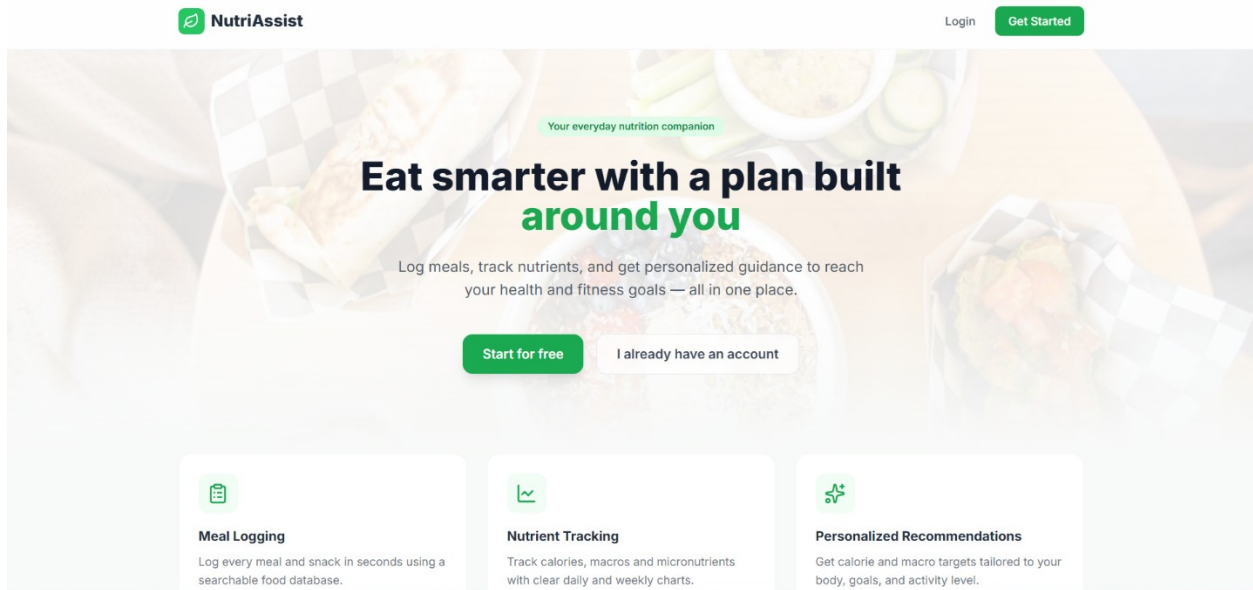
Authentication uses JSON Web Tokens (JWT), not server-side sessions.

- Passwords are hashed with bcrypt (salted, 10 rounds) before being saved — plain-text passwords are never stored.
- On successful register/login, the server signs a JWT containing the user's ID (`jwt.sign({ id }, JWT_SECRET, { expiresIn: "7d" })`) and returns it to the client.
- The frontend stores the token in `localStorage` and attaches it as an `Authorization: Bearer <token>` header on every request via the shared `Axios` instance.
- An `authMiddleware.protect` function verifies the token on every private route, decodes the user ID, loads the user from `MongoDB`, and attaches it to `req.user` for controllers to use.
- Requests with a missing, malformed, or expired token are rejected with a 401 Unauthorized response before reaching the controller.

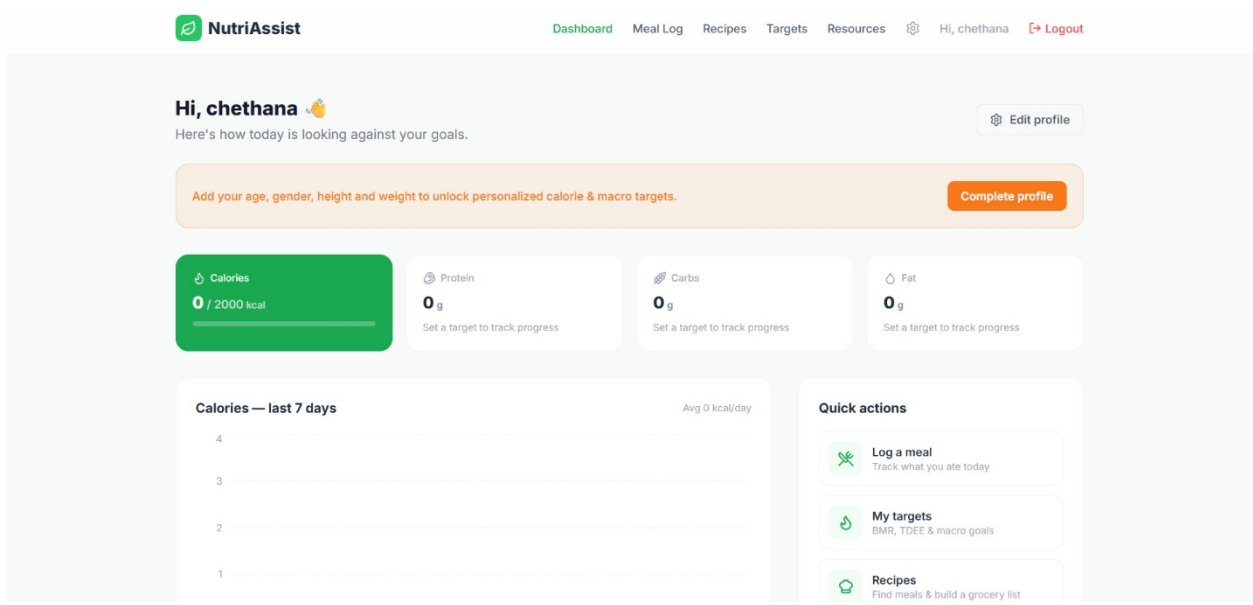
9. User Interface

Selected screens from the application:

Landing page — hero section with feature highlights



Recipes page — filterable suggestions with grocery-list generation



Dashboard Page

Personalized recommendations

We use the Mifflin-St Jeor formula to estimate your BMR, then scale it to your activity level and goal.

Your details

Age

Gender

Height (cm)

Weight (kg)

Activity level

Goal

Your results

🔥 BMR

1133 kcal/day

🔥 TDEE

1756 kcal/day



Daily calorie target

1756 kcal

Macro breakdown



Personalized Recommendations

Meal Log

13-07-2026

Calories

702 kcal

Protein

25 g

Carbs

52 g

Fat

51 g

Log a food

Search foods (e.g. chicken)

Almonds	579 kcal/100g
Apple	52 kcal/100g
Banana	89 kcal/100g
Black Coffee	1 kcal/100g
Broccoli	34 kcal/100g

Breakfast

Almonds
100g · 579 kcal · P21 C22 F50

Banana
100g · 89 kcal · P1.1 C23 F0.3

Lunch

Broccoli
100g · 34 kcal · P2.8 C7 F0.4

Meal Log Page—

10. Testing

The project is currently tested manually. The following approach was used and is recommended going forward:

- Manual functional testing of each user flow (register → login → complete profile → calculate targets → log meals → view dashboard → browse recipes → generate grocery list) after each feature was built.
- API endpoints were exercised directly with a REST client (Postman/Thunder Client) to verify status codes, auth enforcement, and response shapes.
- Cross-browser and responsive/mobile layout checks performed manually in Chrome DevTools device emulation.

Recommended future addition:

- Backend: unit tests for utils/nutritionCalc.js and integration tests for controllers using Jest + Supertest.
- Frontend: component tests with React Testing Library, and end-to-end tests with Playwright or Cypress.

11. Demo

See Section 9 (User Interface) for annotated screenshots of the core screens.

Demo link: <https://drive.google.com/file/d/10TVtsy2BopeAShXOUTjqxEbsnuNB6ddf/view?usp=sharing>

12. Known Issues

- Grocery list aggregation combines ingredients by matching name and unit only; it does not convert between units (e.g. tbsp vs. ml), so the same ingredient logged in different units across recipes will appear as separate lines.
- Recipe suggestions and the nutrition professional directory currently use curated/seeded demo data rather than a live external data source.
- There is no password reset or email verification flow yet.
- No automated test suite exists yet (see Section 10).
- The application has not yet been deployed to a public environment; it currently runs locally only.

13. Future Enhancements

- Deploy the app (e.g. frontend on Vercel, backend on Render, database on MongoDB Atlas) with CI/CD.
- Add an automated test suite (unit, integration, and end-to-end tests).
- Add password reset, email verification, and social login options.
- Unit-aware grocery list consolidation (convert between g/ml/tbsp/tsp automatically).
- Expand the recipe and professional directory datasets, or integrate a live external API.
- Add meal reminders/notifications and a weekly email summary.
- Add an admin panel for managing the food and recipe databases.
- Support barcode scanning for faster food logging.